Midterm Project A: Swing-Up and Balancing of a Furuta Pendulum

COSC 527: Reinforcement Learning

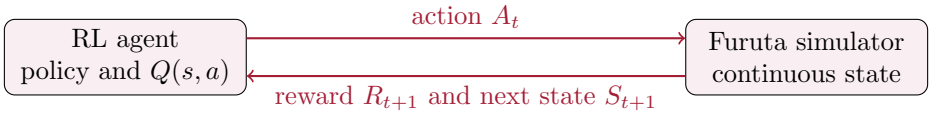
Instructor:	Raffaele Romagnoli
Due date and presentation schedule:	Posted on Canvas
Submission:	Source code, PDF report, and presentation slides

Project objective

In this project, you will use tabular reinforcement learning to address the swing-up and balancing problem for a simulated Furuta pendulum. The pendulum begins near its stable downward equilibrium. The agent must first inject energy through the rotating arm to move the pendulum toward its unstable upright equilibrium and must then keep it upright for five consecutive seconds.

You will transform continuous measurements into a finite state representation, learn an action-value function from sampled interaction, and evaluate the resulting policy without exploration or further learning. Reaching the upright region momentarily is not sufficient: the learned policy must combine energy accumulation during swing-up with stabilization near the upright position.

The purpose is not to derive the pendulum equations. The supplied environment implements the plant dynamics and produces transitions. Your responsibility is to design and analyze the reinforcement-learning agent.



The supplied simulation environment

The continuous state is

$$x = [\phi, \dot{\phi}, \theta, \dot{\theta}]^T,$$

where ϕ is the horizontal-arm angle and θ is the pendulum angle. Because angles that differ by 2π represent the same physical position, use the wrapped angle

$$\widetilde{\theta} = \text{wrapToPi}(\theta) \in [-\pi, \pi].$$

The upright position is $\widetilde{\theta} = 0$, while the downward position is $|\widetilde{\theta}| = \pi$. The environment exposes the interaction

$$(S_t, A_t) \longrightarrow (R_{t+1}, S_{t+1}, \texttt{done}).$$

The default discrete action set applies one of three motor torques:

$$\mathcal{A} = \{-0.02,\ 0,\ +0.02\} \text{ N m}.$$

The simulator uses a sampling time of $T_s = 0.01$ s. The exact safety limits, maximum episode duration, and baseline reward are defined in the supplied Python file and must be reported in your report.

Initial condition and episode outcome

Each episode begins near the downward equilibrium:

$$\theta_0 = \pi + \alpha, \qquad \alpha \sim \mathcal{U}(-5^\circ, 5^\circ),$$

with small randomized initial angular velocities. Randomization on both sides of π prevents the learned policy from depending on only one initial swing direction.

Define the upright region by

$$|\widetilde{\theta}_t| \leq 20^\circ.$$

A successful episode requires the pendulum to remain in this region for five consecutive seconds. Since $T_s = 0.01$ s, success requires

$$N_{\text{hold}} = \frac{5}{0.01} = 500$$

consecutive simulation steps in the upright region. The counter is reset to zero whenever the pendulum leaves the region.

An episode ends when one of the following occurs:

- **success:** the pendulum remains within 20° of upright for 500 consecutive steps;
- **unsafe termination:** a specified arm-position or velocity safety limit is violated; or
- **time-limit truncation:** the maximum episode duration is reached before success.

Being outside the upright region does not terminate the episode, because the pendulum begins outside that region and must be allowed to swing up.

Baseline reward

The reward must provide information during both swing-up and balancing. The supplied baseline uses the wrapped angle and penalizes excessive arm motion and control effort. Its general form is

$$R_{t+1} = \cos(\widetilde{\theta}_{t+1}) - \lambda_\phi \phi_{t+1}^2 - \lambda_u u_t^2,$$

with an additional success bonus. Thus, the angular contribution is $+1$ at upright and -1 at the downward equilibrium. You may study an alternative reward in the controlled comparison, but you must retain baseline results.

Model interpretation. The simulator uses a mathematical model to generate transitions. Your SARSA or Q-learning agent is nevertheless model-free if it learns only from observed tuples $(S_t, A_t, R_{t+1}, S_{t+1})$ and does not call the dynamics function to predict alternative outcomes.

Parameter note. The numerical parameters originate from the Furuta model in Gäfvert (1998), not from a calibrated QUBE-Servo 3 model. Therefore, all claims in this project concern the supplied simulation, not the physical hardware.

Required work

1. Verify the environment

Before training an agent, demonstrate that you understand the supplied interface.

- Run the example program and identify the four state components.
- Apply every legal action and verify the torque reported by the environment.
- Show one transition that continues the episode and identify the possible success, unsafe-termination, and time-limit outcomes.
- State clearly which quantities are chosen by the agent and which are produced by the environment.

2. Construct a finite state representation

The plant state is continuous, but a Q table requires a finite state set. Select ranges and bin boundaries for all four state components. Explain your choices and report the resulting number of state-action entries.

Your discretization must cover the full wrapped pendulum-angle range and must distinguish several values of pendulum angular velocity. It must represent both the large motions needed for swing-up and the smaller motions relevant to balancing. Avoid a table so large that most entries are never visited. Test the mapping on values below, inside, and above every selected range. Values outside the interior bin boundaries must still map to valid table indices.

3. Implement action-value control

Implement both of the following one-step methods:

SARSA (on-policy)	$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$
Q-learning (off-policy)	$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$

For a terminal transition, use continuation value zero. Use an ϵ-greedy behavior policy during training and document how ties among maximizing actions are handled. Initialize the Q table explicitly and state the selected values of α, γ, and ϵ.

Your program must make the difference between the two targets visible in the code. Copying one implementation and changing only its label does not satisfy this requirement.

4. Separate training from evaluation

During training, the agent may explore and must update its Q table. During evaluation:

- freeze the learned Q table;
- disable exploration and use a greedy policy;
- do not update any learned quantity; and
- evaluate multiple episodes from controlled initial conditions.

Use random seeds and report them. Training return is not by itself an unbiased measurement of the final policy because it includes exploration and a changing Q table.

5. Perform controlled experiments

For both SARSA and Q-learning, plot at least:

- episode return during training, with a moving average;
- episode duration and success rate during training;
- wrapped pendulum angle versus time for representative evaluation episodes; and
- the fraction of evaluation episodes that achieve five seconds of continuous balancing.

Evaluate each final policy over at least 20 episodes and report its success rate, mean and standard deviation of return, time to first enter the upright region, and episode duration. Use the same evaluation initial conditions or seeds for the two methods. If an episode never reaches upright, record that outcome explicitly rather than discarding it.

Perform one additional controlled comparison. Change exactly one major design choice, such as the state discretization, exploration schedule, learning rate, or reward definition. Explain why you selected it and interpret the observed effect. Do not change several elements simultaneously and attribute the result to only one of them.

6. Interpret the learned behavior

Answer the following questions in your own words:

(a) Why is this agent considered model-free even though training takes place inside a model-based simulator?

(b) What is the target policy for SARSA? What is the target policy for Q-learning? How can their update targets differ for the same sampled transition?

(c) Which state variables and regions of the Q table were visited most frequently? What does this reveal about your discretization?

(d) Did a larger training return always produce a better greedy evaluation policy? Explain using your results.

(e) How does your reward encourage energy accumulation during swing-up while still favoring stabilization near upright?

(f) Identify one limitation that would matter when transferring the learned controller to a physical QUBE-Servo 3.

Implementation requirements

Your submission must:

- keep the supplied environment separate from the learning agent;
- compute the policy from experience rather than hard-code actions;
- allow the main hyperparameters and bin boundaries to be changed;
- save or otherwise preserve the learned Q tables used for evaluation;
- produce plots and summary statistics programmatically; and
- include instructions that reproduce the reported experiments.

You may make small, documented corrections to the supplied environment. Do not replace the plant dynamics or termination conditions without prior approval. You may modify the reward as the single controlled comparison, but you must retain and report results for the supplied baseline reward.

Deliverables

Written report (4–6 pages, excluding appendices)

The report should contain:

- a concise problem and environment description;
- the selected state discretization and table size;
- the behavior policy, update rules, hyperparameters, and stopping criterion;
- training and evaluation results with labeled plots;
- the controlled comparison and its interpretation;
- answers to the conceptual questions; and
- limitations and lessons learned.

Source code

Submit clean source files, the final learned tables or fixed random seeds needed to reproduce them, and a short README containing installation and execution instructions. Do not submit generated virtual environments or large temporary files.

Presentation and demonstration (10 minutes)

Present the problem formulation, design decisions, learning results, and one evaluation demonstration. The presentation will be followed by approximately three minutes for questions and transition. Design the presentation to fit the assigned time; presentations may be stopped when the ten-minute limit is reached. Every student must be prepared to explain the implementation and answer questions about the update rule, exploration, reward, and evaluation procedure.

Grading rubric

Component	Weight	Primary criteria
Written report	30%	Completeness (10%), technical correctness (10%), clarity and interpretation (10%)
Presentation and demonstration	30%	Completeness (10%), understanding and correctness (10%), clarity and time management (10%)
Implementation and results	40%	Correct environment interaction and discretization (10%), correct SARSA and Q-learning updates (15%), reproducible swing-up and balancing experiments (10%), code clarity (5%)

Correctness includes the ability to explain and defend the submitted work. A program that runs but does not implement the stated update, or results that cannot be reproduced, will not receive full implementation credit. Successful swing-up on every evaluation trial is not an all-or-nothing grading condition. Credit is based on the correctness of the learning procedure, quality of the experimental evidence, and interpretation of both successful and unsuccessful trials. A claimed successful controller must, however, satisfy the stated five-second balancing criterion.

Use of generative AI

Use of generative AI tools is permitted only in accordance with the course syllabus. If you use such a tool, include an appendix containing:

- the relevant prompts or questions submitted to the tool;
- a brief description of how the responses influenced your work;
- at least one suggestion that you verified, corrected, modified, or rejected; and
- the portions of the code, report, or presentation for which the tool provided assistance.

A complete transcript is not required unless specifically requested. You are responsible for understanding, verifying, and explaining every part of the submission. The presentation may include questions that require you to connect your code to the corresponding reinforcement-learning equations.

Provided material and optional reference

The project package provides the Python environment, a minimal interaction example, installation requirements, and a short model guide. The technical derivation is available as an optional reference:

> M. Gäfvert, *Modelling the Furuta Pendulum*, Technical Report TFRT-7574, Department of Automatic Control, Lund Institute of Technology, 1998.

Consult Canvas for the authoritative project package, submission instructions, deadlines, presentation schedule, and any announced changes.